Karger's Algorithm

Isha Patel, Aurelia Peterson Rajalingam, Kritika Pandit

1 Introduction

Karger's Algorithm is a Monte Carlo algorithm for finding the minimum cut of an undirected, weighted graph, the smallest set of edges that disconnects the graph into two components. Unlike deterministic approaches like the Max-Flow Min-Cut theorem, Karger's algorithm employs random edge contractions until only two vertices remain, at which point the remaining edges define a potential minimum cut. This problem is fundamental in graph theory and has applications in network reliability, clustering, and parallel computing.

1.1 History and Significance of Karger's Algorithm

Developed in 1993 by David Karger, this Monte Carlo algorithm relies on random sampling to approximate the actual minimum cut, with a small probability of error. The term "Monte Carlo" originates from the famous casino in Monaco, symbolizing the gambling nature of Monte Carlo algorithms. To improve accuracy, the algorithm is run multiple times, and the smallest cut found across all runs is selected. Karger's work influenced further developments in graph partitioning, clustering, and network optimization. Its randomized edge contraction method makes it easy to implement and adapt for various applications.

1.2 Real-World Applications

1. **Network Design and Reliability** - Identifies weak points in communication and power networks for fault-tolerant system design.
2. **Clustering in Social and Biological Networks** - Detects communities in social networks and clusters in protein interaction networks.
3. **Image Segmentation** - Uses minimum cut to partition images into meaningful segments based on affinity.
4. **Circuit Design** - Helps efficiently partition circuits for VLSI hardware design and layout optimization.

2 The Algorithm

2.1 Understanding the Minimum Cut Problem

Given an undirected, connected graph $G = (V, E)$, a cut partitions the vertex set V into two disjoint subsets. The minimum cut problem seeks the partition with the fewest crossing edges.

2.2 Pseudocode

Algorithm 1 Karger's Algorithm for Minimum Cut

Require: Graph $G = (V, E)$
Ensure: A minimum cut of G
1: **while** $|V| > 2$ **do**
2: Select a random edge $(u, v) \in E$
3: Contract (u, v) by merging u and v into a single vertex
4: Remove self-loops
5: **end while**
6: **Return** the number of edges between the two remaining vertices

Explanation of Steps

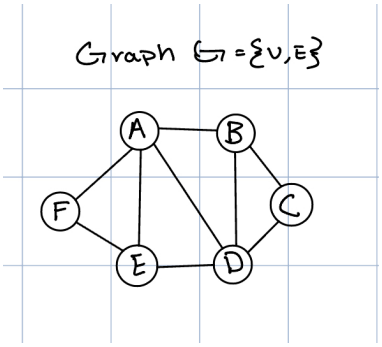
1. Select a random edge (u, v).
2. Contract (u, v) by merging u and v into a super-node.
3. Remove self-loops.
4. Repeat until only two vertices remain.
5. The remaining edges between them form a possible minimum cut.

Since the algorithm is randomized, it is executed multiple times to increase accuracy.

2.3 Example Execution

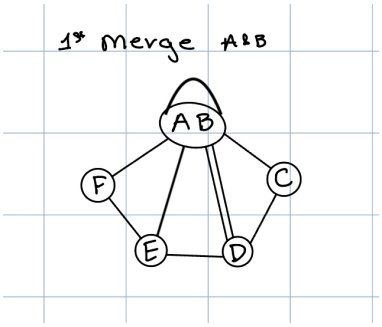
Consider a graph with vertices $V = \{A, B, C, D, E, F\}$ and edges

$$E = \{(A, B), (A, C), (B, C), (C, D), (C, E), (D, E), (E, F)\}.$$

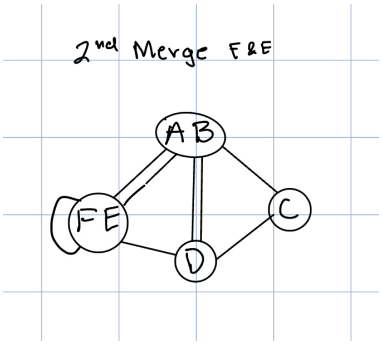


Step-by-Step Execution

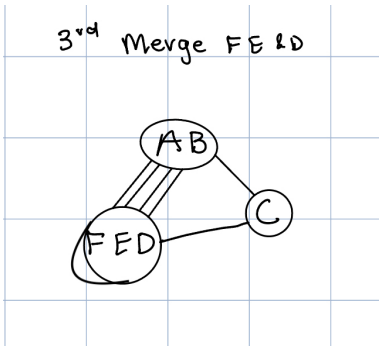
- Randomly pick and contract edges, this time we are going to choose (A, B) which will form the supernode AB.



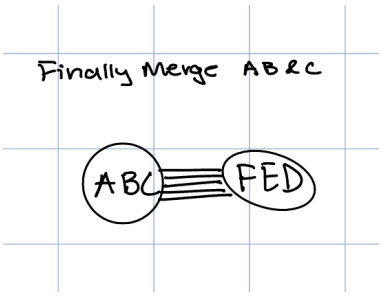
- Remove self-loops and repeat with (F, E) to form FE.



- Repeat previous steps and this time contract (FE, D) into supernode FED.



- Repeat previous steps and this time contract (AB, C) into supernode ABC, leaving two vertices: ABC and FED. The self-loop on ABC was removed.



The number of edges between ABC and FED is the computed minimum cut, which in this case is 5.

2.4 Probability

The probability of identifying the minimum cut by choosing a random cut in the graph G is as small as $2^{-\Omega(n)}$. However, using Karger's algorithm the probability is $\frac{2}{n(n-1)}$.

2.5 Amplified Karger's

A general trick for increasing the success rate of Monte Carlo algorithms is: If an algorithm has a success probability of P, we can run it $C \cdot \frac{\ln n}{P}$ times and take the best result found. This ensures that the probability of failure is at most $\frac{1}{n^C}$, making the algorithm more reliable. Let us leverage this information and create an algorithm called Amplified Karger's. In Amplified Karger's we run Karger's $C \cdot \binom{n}{2} \ln n$ where $C > 0$ and keep track of the minimum result of these runs.

3 Proof

3.1 Proof that Karger's Algorithm returns an actual cut of the graph G

Claim: Karger's Algorithm returns an actual cut of the graph G

Proof: The algorithm starts with an undirected, unweighted graph G consisting of vertex set V and edge set E.The algorithm repeatedly contracts edges, merging the two vertices at the ends of each contracted edge into a "supernode." After each contraction, one edge is removed, and the number of vertices decreases by one. At the end of the algorithm, you end up with two supernodes. These supernodes are composed of the original vertices from G, and they form a partition of the graph. Each original vertex is in one of the two supernodes.

Once the algorithm completes, only the edges between vertices in different supernodes remain. The edges within each supernode have been removed during the contraction process. The remaining edges connect vertices from one supernode to vertices in the other. By definition, the edges left after the algorithm are the only ones connecting the two sets of the partition. Therefore, they form a valid cut-set of the graph.

3.2 Proof that Karger's Algorithm returns the minimum cut with probability $\frac{2}{n(n-1)}$

We know that the total number of vertices in a graph is equal to $2|E|$. Summing the degree of all the vertices in the graph gives us $2|E|$.

$$\sum_{v \in V} deg(v) = 2|E|$$

Claim: If there are n vertices, the average degree of a vertex is $\frac{2|E|}{n}$

Proof: Since, Karger's is a randomized algorithm. We need to analyze the average degree of a vertex in our graph. If you pick a vertex V' at random from n vertices, $P(V' = u) = \frac{1}{n}$ as the probability of picking each vertex is equally likely and there are n vertices. Expectation is calculated via the formula

$$E[g(X)] = \sum_{v \in V} g(v)P(X = v)$$

Substituting $deg(v)$ in $g(X)$ we get:

$$E[deg(V')] = \sum_{v \in V} deg(v) P(V' = v) = \frac{1}{n} \sum_{v \in V} deg(v) = \frac{2|E|}{n}$$

Using our knowledge about expectation and the sum of all the degrees in a graph, we can conclude that the average degree of a vertex is $\frac{2|E|}{n}$.

Claim: The size of the minimum cut is at most $\frac{2|E|}{n}$

Proof: A cut in a graph is a way of splitting the Vertex set V into two disjoint subsets. One way to define a cut is to put a single vertex v in the first partition set. The other partition set would contain the remaining $n-1$ vertices. The size of this cut would be the number of edges leaving vertex v. This is $deg(v)$. Since the minimum cut is the smallest possible cut in the graph, it cannot be larger than this cut. Therefore,

$$\text{(size of minimum cut)} \leq deg(v) \leq \frac{2|E|}{n}$$

Claim: Assuming the size of the minimum cut is at most $\frac{2|E|}{n}$, the probability of picking an edge in the minimum cut is $\frac{2}{n}$

Proof: Assume, the size of the minimum cut is at most $\frac{2|E|}{n}$. To calculate the probability of selecting an edge in the minimum cut divide the size of the minimum cut by the number of edges in the graph.

$$\frac{\frac{2|E|}{n}}{|E|} = \frac{2}{n}$$

Claim: Assuming Karger's picks an edge at random, the probability that the cut lies across the minimum cut is $\frac{2}{n(n-1)}$

Proof: Krager's returns the actual minimum cut as long it never picks an edge across the minimum cut to contract. If it always picks a non-cut edge, then this edge will connect two nodes on the same side of the cut, and so it is okay to contract them together. Using probability principles:

P(Krager's outputs min cut) = P(1st edge selected is not min cut)P(2nd selected is not min cut) ...P(n - 1 edge selected is not mini cut)

The probability that an edge selected is not across the min cut is equal to $1 - \frac{2}{n}$. This is because $P(A^c) = 1 - P(A)$. We proved that P(selecting edge in min cut) = $\frac{2}{n}$. Each time an edge is collapsed, the number of nodes decreases by 1. Therefore:

$$\begin{aligned} \text{P(Krager's outputs min cut)} &= (1 - \tfrac{2}{n})(1 - \tfrac{2}{n-1})(1 - \tfrac{2}{n-2})...(\tfrac{2}{4})(\tfrac{1}{3}) \\ &= (\tfrac{n-2}{n})(\tfrac{n-3}{n-1})...(\tfrac{2}{4})(\tfrac{1}{3}) \\ &= \tfrac{2}{n(n-1)} \end{aligned}$$

Proof that Amplified Karger returns the minimum cut with probability of at least $1 - \frac{1}{n^C}$.

Claim: The probability that the amplified algorithm correctly finds the minimum cut is at least: $1 - \frac{1}{n^C}$, where C is a constant used in amplification.

Proof: The probability that Amplified Karger fails is the probability that Karger's algorithm fails in all $C \cdot \binom{n}{2} \ln n$ independent runs is:

$$P(\text{Amplified Karger fails}) = (P(\text{Karger fails}))^{C \cdot \binom{n}{2} \ln n}.$$

Since the probability of failure in a single run of Karger's algorithm is at most $1 - \frac{2}{n(n-1)} = 1 - \frac{1}{\binom{n}{2}}$, we get:

$$\left(1 - \frac{1}{\binom{n}{2}}\right)^{C\binom{n}{2} \ln n}.$$

Using the fact that $(1 - \frac{1}{x})^x \leq \frac{1}{e}$, we can bound this as:

$$\left(\frac{1}{e}\right)^{C \ln n} = \frac{1}{n^C}.$$

Thus, the probability that Amplified Karger succeeds is at least:

$$1 - \frac{1}{n^C}.$$

4 Runtime Analysis

4.1 Runtime of One Iteration of Karger's:

The while loop in Karger's Algorithm runs until the number of vertices in Graph G is equal to 2. Each edge contraction reduces the number of vertices by 1 since it merges two vertices into one. Therefore, this while loop runs exactly $(n - 2)$ times where $n = |V|$.

In each iteration of the while loop, the algorithm selects a random edge in the graph, contracts that edge, removing any self-loops formed in the process. The selection of a random edge is an $O(1)$ operation. At the end of the algorithm, the time to count the remaining vertices depends on the length of the adjacency lists. In the worst case, where the graph is dense, it takes $O(n)$, as it requires iterating through one of the adjacency lists, which is of length $O(deg(V))$. In the average case, the average degree of a vertex is $\frac{2|E|}{n}$, so this operation takes $O(\frac{|E|}{n})$ time.

Breaking down the contraction process, if the edge (u, v) is being contracted, all edges incident to v are added to u, and self-loops are deleted. If the graph is represented as an adjacency list, updating the adjacency structure requires iterating over the edges of v and

transferring them to u, which takes $O(\deg(V))$ time. In the worst case, for a dense graph, this can take $O(n)$ time per contraction, while in the average case, it takes $O(\frac{|E|}{n})$ time.

Since there are $O(n)$ contractions in a single execution, the total time spent on the algorithm in the worst case is $T(n) = (n * n) + n$, which is in the time complexity $O(n^2)$. On average however, $T(n) = (n * \frac{|E|}{n}) + \frac{E}{n}$ which is in the time complexity $O(|E|)$. Therefore, Karger's Algorithm has a worst-case time complexity of $O(n)$ for dense graphs and an average-case time complexity of $O(|E|)$, where $|E|$ is the number of edges in the graph.

4.2 Runtime of Multiple Iterations

Since each run of Karger's algorithm takes $O(n^2)$ time worst case, and we run it $\frac{Cn^2 \log n}{2}$ times, $\mathrm{T(n)} = n^2 * \frac{Cn^2 \log n}{2} = \frac{C}{2} * n^4 \log n$. This is in the time complexity, $O(n^4 \log n)$.

References

[1] GeeksforGeeks. *Introduction and Implementation of Karger's Algorithm for Minimum Cut*. https://www.geeksforgeeks.org/introduction-and-implementation-of-kargers-algorithm-for-minimum-cut/

[2] Kleinberg, Jon, and Éva Tardos. *Algorithm Design*. Pearson, 2006. pp. 714-719.

[3] Mopidevi, Lohith. *Exploring the Karger Algorithm for Graph Min-Cut with Real-Life Applications*. Medium, 18 July 2023. https://medium.com/@lohith_mopidevi/exploring-the-karger-algorithm-for-graph-min-cut-with-real-life-applications-1d4e48e

[4] *Beauty of Karger's Algorithm*. Medium, 20 May 2022. https://medium.com/data-science/beauty-of-kargers-algorithm-6de7e923874a

[5] Princeton University. *COS521: Advanced Algorithm Design – Lecture 2: Randomized Contractions*. Princeton University, Fall 2013. https://www.cs.princeton.edu/courses/archive/fall13/cos521/lecnotes/lec2final.pdf

[6] Princeton University. *COS521: Advanced Algorithm Design – Lecture 1: Introduction to Randomized Algorithms*. Princeton University, Fall 2022. https://www.cs.princeton.edu/~hy2/teaching/fall22-cos521/notes/lec1.pdf

[7] Stanford University. *CS161: Design and Analysis of Algorithms – Lecture 15: Minimum Cut and Maximum Flow*. Stanford University, 2016. https://web.stanford.edu/class/archive/cs/cs161/cs161.1166/lectures/lecture15.pdf